```
      expect(mail.to).to eq(["monitor@tasks.com"])
      expect(mail.body.encoded).to match(/Learn how to test mailers/)
    end
  end
end
```

In this case you're not actually sending the email through the Rails mailer; you're just generating an email object and testing its values.

Outside of core Rails, the email-spec library provides a number of useful helpers.[6] For the most part they are ways of performing the tests you've already examined, but with a slightly cleaner syntax. The library also provides the ability to follow a link in an email back to the its source, which is helpful for acceptance-testing of user interactions that include email.

Testing Views and View Markup

You've tested a helper for project status, but when you go to the browser the new status DOM elements don't show up. This is because you haven't placed your new helper in the view template itself. Naturally, you would like your dazzling two-line helper to be incorporated into the view. From a TDD perspective, you have a few options:

- Write no further tests; just insert the helper into the view template. Technically you're not adding logic, so you can kind of squeak by with this one. I don't mean to be glib here—having no extra test may be the right choice when the test is expensive, trivial in the larger scheme of things, and easy to visually inspect.
- Write an integration test using Capybara, as you saw in Chapter 3, *Test-Driven Rails*, on page 41, and in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151. If you've been using outside-in development, you may already have an integration test in place.
- Write a Rails view test. This has the advantage of being somewhat faster than the integration test, and you may be able to piggyback it on existing controller tests.

RSpec allows you to specify view tests independent of controllers. (Though you can get views to run from controller tests, it's not the default and it's not recommended.) The RSpec convention is to place view tests in the spec/views folder, with one spec file to a view file, so the view in app/views/projects/index.html.erb is specified in spec/views/projects/index.html.erb_spec.rb.

6. http://github.com/bmabey/email-spec